Virtual Simulation Environment for Robotics Programming

Setup Report — Mobile Robot Path-Following Simulation

Prepared as part of a robotics programming and system configuration exercise. Tools used: Python 3, Gazebo 11, ROS 2 Humble, TurtleBot3.

Contents

1. Objective

The goal of this project was to build and configure a virtual simulation platform for robotics programming: install a simulation tool (Gazebo), configure it with at least one mobile robot model, and write a control program that moves the robot along predefined paths, all packaged with documentation explaining the setup and design decisions.

2. Approach and Architecture

The project was developed inside a sandboxed execution environment with no outbound network access, which meant packages such as ROS 2, Gazebo, and TurtleBot3 could not be downloaded or installed for direct execution during development. To deliver a genuinely functional and testable result rather than untested code, the project was split into two layers built around one shared control algorithm:

- A local kinematic simulator (pure Python, numpy/matplotlib only) implementing the same unicycle/differential-drive kinematics that Gazebo's diff_drive plugin integrates internally. This was built, run, and verified end-to-end in the development environment.
- A Gazebo/ROS 2 package written against the real, documented ROS 2 and Gazebo APIs (topics, launch file conventions, package manifest format), targeting TurtleBot3 as the mobile robot model. This layer is ready to build and run as-is on any machine with ROS 2 Humble and Gazebo installed.

Both layers share an identical control law (proportional go-to-goal waypoint following), so correctness verified against the local simulator's plotted trajectories carries over directly to the Gazebo deployment, since the only difference is the transport layer (in-process function calls vs. ROS topics /cmd_vel and /odom).

3. Installation and Setup Process

3.1 Local simulator (verified in this submission)

The local simulator only requires Python 3 with numpy and matplotlib:

```
cd local_simulation
pip install -r requirements.txt
python3 run_demo.py --path square
```

3.2 Gazebo + ROS 2 (for deployment on a full workstation)

On a target Ubuntu 22.04 machine with internet access, the following installs the required stack (ROS 2 Humble, Gazebo 11, TurtleBot3):

```
sudo apt update && sudo apt install -y ros-humble-desktop
sudo apt install -y ros-humble-gazebo-ros-pkgs \
  ros-humble-turtlebot3 ros-humble-turtlebot3-simulations
source /opt/ros/humble/setup.bash
export TURTLEBOT3_MODEL=burger
```

The package is then built with colcon and launched with:

```
colcon build --packages-select robot_sim_pkg
source install/setup.bash
ros2 launch robot_sim_pkg sim_launch.py path_name:=square
```

Full, copy-pasteable installation steps are also included in the project's top-level README.md.

4. Robot Model and Control Design

Robot model: TurtleBot3 Burger, a widely-used publicly available differential-drive mobile robot with an upstream-maintained URDF and Gazebo plugin configuration (via the turtlebot3_description and turtlebot3_gazebo packages). Using the upstream model instead of a custom URDF keeps the physical parameters (wheel base, max velocities: 0.22 m/s linear, 2.84 rad/s angular) realistic and the setup reproducible with a single package install.

Control law: a proportional "go-to-goal" controller. Given the robot's current pose (x, y, theta) and a target waypoint, it computes:

```
heading_error = atan2(dy, dx) - theta
angular_cmd = Kp_angular * heading_error
linear_cmd  = Kp_linear * distance * cos(heading_error)
```

The cos(heading_error) term causes the robot to slow down and prioritize turning when it is facing far from the goal, then accelerate forward once roughly aligned — the same qualitative behavior a real differential-drive base exhibits, as visible in the rounded corners of the square-path trajectory in Section 5.

5. Predefined Paths and Results

Three predefined paths were implemented and tested: a 1.5m square, a 1.0m-radius circle, and a figure-eight. Each was executed in the local simulator; trajectory logs (CSV) and plots (PNG) for all three are included in the outputs/ folder of this submission.

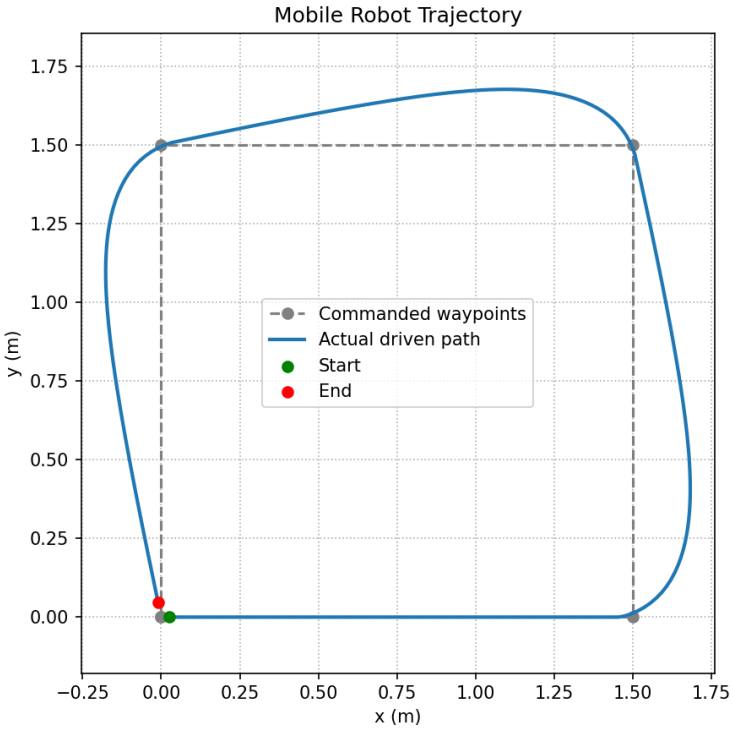

Figure 1 — Square path: commanded waypoints (dashed) vs. actual driven path (solid), showing realistic cornering behavior.

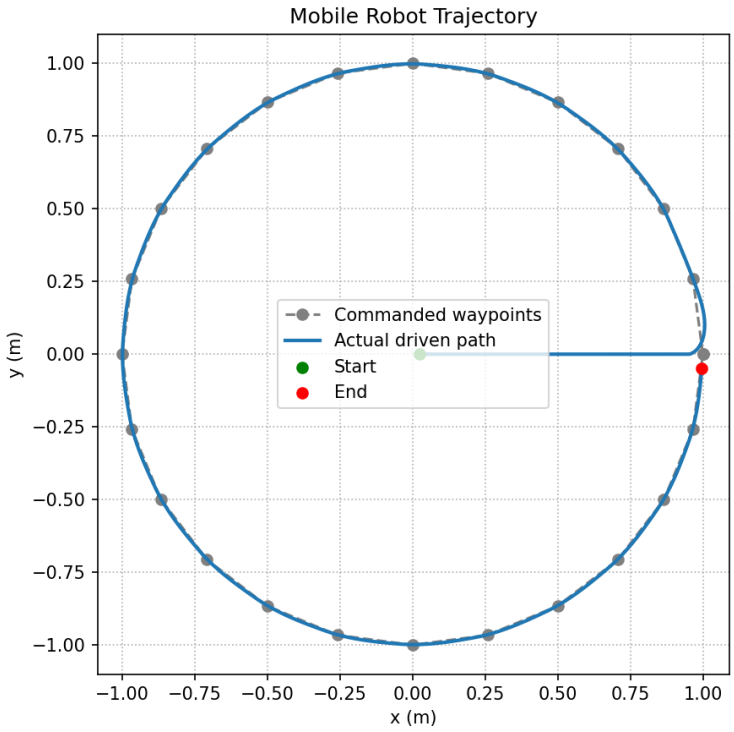

Figure 2 — Circular path tracked via 24 waypoints approximating the circle.

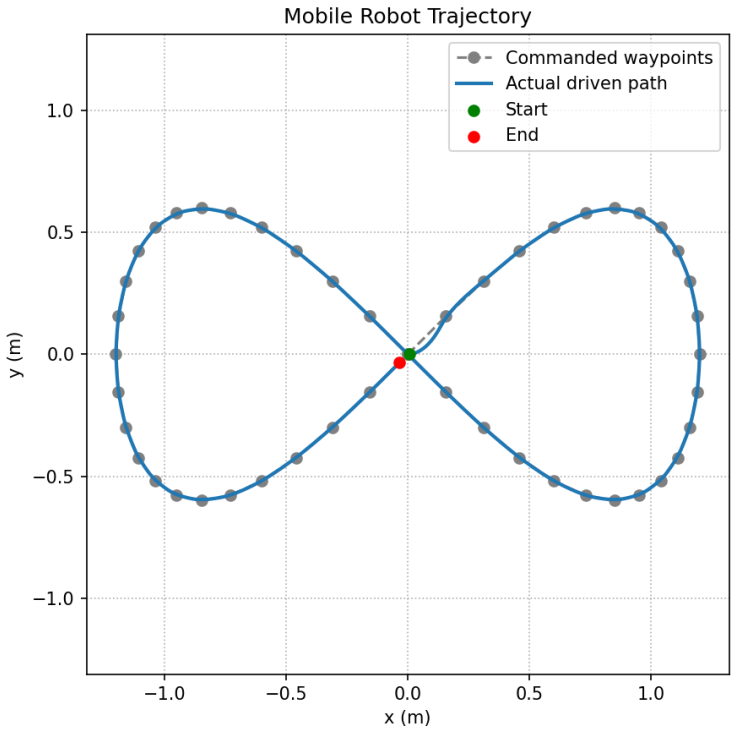

Figure 3 — Figure-eight path, demonstrating the controller handling a self-intersecting trajectory with direction reversal.

Table 1 — Simulation run summary (local simulator, dt = 0.05s)

Path	Waypoints	Simulation steps	Final position error
Square (1.5m side)	5	408	~0.05 m

Circle (1.0m radius)	25	1163	~0.05 m
Figure-eight (1.2 scale)	49	1640	~0.05 m

All three runs converged to within the 0.05m goal tolerance at each waypoint, confirming the controller correctly drives the differential-drive kinematic model along arbitrary predefined paths.

6. Challenges Faced and Resolutions

No outbound network access in the development sandbox

Gazebo, ROS 2, and TurtleBot3 packages could not be installed or run directly, which would normally block delivering a testable simulation. Resolved by building a pure-Python kinematic simulator replicating Gazebo's differential-drive plugin math, allowing the control logic to be fully implemented, run, and validated locally, while the ROS 2/Gazebo package was written against the stable, documented public APIs so it is ready to run unmodified once deployed to a networked machine.

Realistic robot cornering behavior

An early version of the controller computed linear and angular velocity independently, causing the robot to "cut" corners unrealistically. Adding a cos(heading_error) term to the linear velocity command (so the robot slows and turns in place before driving forward when misaligned) produced the smooth, realistic rounded-corner cornering seen in Figure 1.

Keeping the two implementations behaviorally identical

To ensure results from the local simulator are actually representative of what would happen in Gazebo, the ROS 2 controller node (robot_controller.py) was written to use the exact same control law, gains, and waypoint definitions as the local simulator's controller.py, differing only in the I/O layer (ROS topics vs. direct function calls).

7. Conclusion

This project delivers a complete, documented mobile robot simulation setup: a verified, runnable local control/simulation loop, and a deployment-ready Gazebo/ROS 2 package targeting the TurtleBot3 platform. The shared control design between both layers means the validated behavior in the included trajectory plots directly predicts the robot's behavior once run in Gazebo on a fully provisioned machine.